Laboratório de Conjuntos

Atividade prática: Teoria dos Conjuntos e Conjuntos Numéricos

Curso: Desenvolvimento de Software Multiplataforma – Fatec Osasco

Apresentação

Nesta atividade em equipe, a matemática aparece como ferramenta para resolver problemas de programação. Ela tem três níveis de dificuldade e contempla os conteúdos já estudados: Teoria dos Conjuntos e Conjuntos Numéricos. Em todas as partes, cada equipe deve demonstrar o funcionamento do seu programa com exemplos de execução.

Formato

- **Equipes:** duplas ou trios.
- **Duração:** 2 a 3 aulas.
- **Linguagem:** Python (ou JavaScript/Java, ou outra). Em Python, os tipos `set` e `frozenset` já modelam conjuntos, o que ajuda muito.

Parte 1: Teoria dos Conjuntos (básico)

Desenvolver um programa de menu que permita ao usuário informar dois conjuntos A e B e escolher operações:

- União, interseção, diferença (A−B e B−A) e diferença simétrica.
- Verificação de pertinência ($x \in A$) e de inclusão ($A \subseteq B$, $A \subset B$).
- Cardinalidade e complemento de A em relação a um conjunto universo U.

Desafio: não usar os operadores prontos da linguagem. Implementar as operações com laços e condicionais e, depois, comparar com a solução nativa para validar.

Exemplos de execução (obrigatório): apresentar, no mínimo, três execuções diferentes, sendo uma delas com conjunto vazio, conjuntos iguais ou conjuntos disjuntos, e mostrar a comparação entre a sua implementação e a solução nativa.

Parte 2: Conjuntos Numéricos (intermediário)

Desenvolver um classificador de números. Dado um número (inteiro, fração, decimal ou raiz), o programa informa a quais conjuntos ele pertence (N, Z, Q, I, R) e justifica a classificação.

- Entrada de frações no formato a/b e de números decimais.
- Identificação de dízimas periódicas e cálculo da fração geratriz.
- **Bônus:** geração de todos os primos de N até n (crivo de Eratóstenes) e listagem dos divisores de um número.

Exemplos de execução (obrigatório): apresentar, no mínimo, oito números de entrada, incluindo um inteiro negativo, uma fração, um decimal exato, uma dízima periódica e uma raiz irracional.

Parte 3: Integração (avançado)

Cada grupo escolhe um dos mini-projetos abaixo:

1. **Sistema de recomendação simples:** cada usuário tem um conjunto de interesses, e o programa usa interseção e união para calcular a afinidade (índice de Jaccard = $|A \cap B| / |A \cup B|$).
2. **Controle de acesso por perfis:** permissões representadas como conjuntos, com verificação de quem pode o quê (subconjuntos, união de perfis).
3. **Produto cartesiano e conjunto das partes:** gerador de combinações, como configurações de um produto ou casos de teste, com análise do crescimento (2^n) e discussão sobre desempenho.
4. **Intervalos reais:** classe que representa intervalos como [2, 5) e implementa união, interseção e pertinência.

Exemplos de execução (obrigatório): apresentar, no mínimo, três cenários de uso do mini-projeto escolhido, com os resultados esperados calculados à mão e comparados com a saída do programa.

Entregáveis

- Código no GitHub, com README explicando as decisões tomadas.
- Exemplos de execução, em prints de tela ou logs no README, para cada uma das três partes, conforme as exigências indicadas acima.
- Casos de teste, incluindo conjunto vazio, conjuntos iguais, conjuntos disjuntos e elementos repetidos na entrada.
- Relatório de 1 página, respondendo: onde a teoria dos conjuntos apareceu no código? Qual operação foi mais difícil de implementar e por quê?

Critérios de avaliação (sugestão)

Critério	Peso
Correção das operações e dos casos de teste	35%
Qualidade do código (organização, nomes, funções)	25%
Parte 3 (criatividade e conexão com a teoria)	25%
Relatório e uso do Git	15%

Exemplos que podem ser usados

Os exemplos abaixo são sugestões para testar e demonstrar os programas. Os resultados indicados servem de gabarito para conferir a saída. Além deles, cada equipe deve incluir ao menos um exemplo criado por ela.

Parte 1: operações com conjuntos

Exemplo 1: caso geral. A entrada tem elementos repetidos, que o programa deve descartar.

Conjunto A: 1, 2, 2, 3, 4 Conjunto B: 3, 4, 5, 6 Universo U: 1 a 10

```
A = {1, 2, 3, 4}          |A| = 4
B = {3, 4, 5, 6}          |B| = 4
A ∪ B = {1, 2, 3, 4, 5, 6}
A ∩ B = {3, 4}
A − B = {1, 2}
B − A = {5, 6}
A Δ B = {1, 2, 5, 6}
Complemento de A em U = {5, 6, 7, 8, 9, 10}
3 ∈ A ? Verdadeiro        5 ∈ A ? Falso
{3, 4} ⊆ A ? Verdadeiro   {3, 4} ⊂ A ? Verdadeiro
```

Exemplo 2: casos de borda. Cada linha deve ser testada pela equipe.

Situação	Entrada	Resultado esperado
Conjunto vazio	A = ∅, B = {1, 2}	A ∪ B = {1, 2}; A ∩ B = ∅; A ⊆ B é verdadeiro
Conjuntos iguais	A = B = {1, 2, 3}	A − B = ∅; A Δ B = ∅; A ⊆ B é verdadeiro; A ⊂ B é falso
Conjuntos disjuntos	A = {1, 2}, B = {3, 4}	A ∩ B = ∅; A ∪ B = {1, 2, 3, 4}; A Δ B = {1, 2, 3, 4}
Um contido no outro	A = {1, 2}, B = {1, 2, 3}	A ⊂ B é verdadeiro; B ⊆ A é falso; B − A = {3}
Repetidos na entrada	A = 1, 1, 1, 2	A = {1, 2}; |A| = 2

Parte 2: classificador de números

Exemplo de saída esperada para uma dízima periódica:

Entrada: 1,2(5)
Tipo: dízima periódica (parte não periódica: 2; período: 5)
Fração geratriz: 113/90
Pertence a: Q, R
Não pertence a: N, Z, I

Números sugeridos para teste:

Entrada	Pertence a	Observação
7	N, Z, Q, R	Número natural.
−3	Z, Q, R	Inteiro negativo, não é natural.
0	N, Z, Q, R	Definam no README se N inclui o zero.
3/4	Q, R	Fração equivalente ao decimal exato 0,75.
−5/2	Q, R	Fração negativa, equivale a −2,5.
6/3	N, Z, Q, R	A fração simplifica para 2.
0,(3)	Q, R	Dízima periódica, fração geratriz 1/3.
2,(7)	Q, R	Dízima periódica, fração geratriz 25/9.
1,2(5)	Q, R	Dízima periódica composta, geratriz 113/90.
0,(9)	N, Z, Q, R	Caso curioso: a geratriz é 9/9 = 1.
√9	N, Z, Q, R	A raiz é exata e vale 3.
√2	I, R	Raiz não exata, portanto irracional.

Bônus. Primos de N até 30: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29. Divisores de 36: 1, 2, 3, 4, 6, 9, 12, 18, 36.

Parte 3: mini-projetos

Opção 1: recomendação por interesses (índice de Jaccard).

Interesses: Ana = {rock, jazz, cinema, games}; Bia = {jazz, cinema, séries, viagens}; Caio = {games, tecnologia, cinema}.

Par	Interseção	União	Jaccard
Ana e Bia	{jazz, cinema} (2)	6 elementos	2/6 = 0,33
Ana e Caio	{games, cinema} (2)	5 elementos	2/5 = 0,40
Bia e Caio	{cinema} (1)	6 elementos	1/6 = 0,17

Resultado esperado: para a Ana, a recomendação mais forte é o Caio, o de maior afinidade.

Opção 2: controle de acesso por perfis.

Perfis: leitor = {ler}; editor = {ler, editar}; admin = {ler, editar, excluir, gerir_usuarios}; financeiro = {ver_relatorios}.

- Usuário com os perfis leitor e editor: permissões = união = {ler, editar}.
- Usuário com os perfis editor e financeiro: permissões = {ler, editar, ver_relatorios}.
- Esse usuário pode excluir? Não, porque excluir ∉ permissões.
- editor ⊆ admin? Verdadeiro. financeiro ⊆ admin? Falso.